

第一讲 信息系统概述

保护对象：从保护信息切换到保护信息系统

一、什么是信息系统？

1、什么是信息系统（Elsevier 的定义）

Information systems are the software and hardware systems that support data-intensive applications.

2、几类典型的信息系统

管理信息系统、办公自动化系统、电子商务系统、数据处理系统、决策支持系统。

3、More Challenges

（1）实时性

在秒杀、会场等方面都是个性化、千人千面，数据不仅大，而且要做到非常实时。到目前为止，菜鸟物流系统已经产生了超过 10 亿笔的物流单，这个数据还在快速上升，这些都需要依靠阿里云背后大量的计算能力。

（2）数据除了批处理之外，还有流处理

就是实时处理所有数据，每分、每秒都在变，它并不是从数据库里面统计出来的，而是每生成一笔订单，系统自动一层层把数据汇集上来。

（3）数以十万计的物理服务器管理

要把这么多服务器全部管理起来，除了操作系统之外，还要管理所有的消息流转。

二、信息系统架构

1、架构决定性能

2、什么是架构

架构（architecture）是对复杂系统的抽象，目的是帮助人们能够正确、合理地理解、设计和最终构建复杂的系统。

系统的组成部分、它们相互之间的关系、对环境的关系和指导设计以及演进原则。

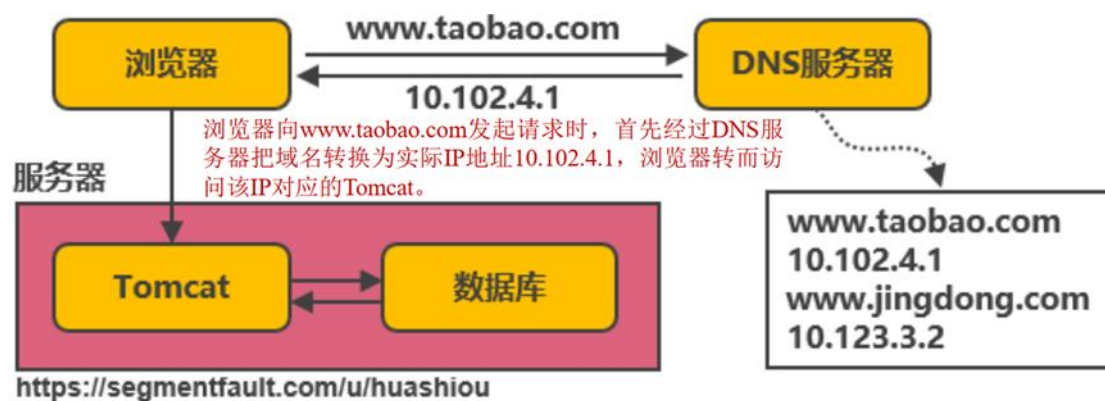
3、信息系统体系架构分析的必要性

从系统分析方法论的角度看，如果一个系统过于复杂，则对其进行分析的最好方法就是按照某种角度对整个系统进行解构。

三、信息系统架构演进

1、Evolution of Architecture：单机架构

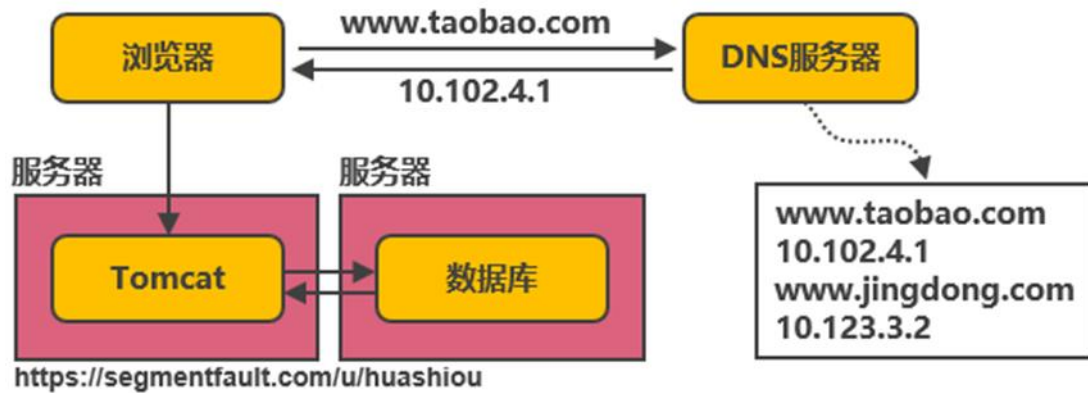
在网站开设初期，应用数量与用户数都较少，可以把 Tomcat 和数据库部署在同一台服务器上。



随着用户数的增长，Tomcat 和数据库之间竞争资源，单机性能不足以支撑业务。

2、第一次演进：Tomcat 与数据库分开部署

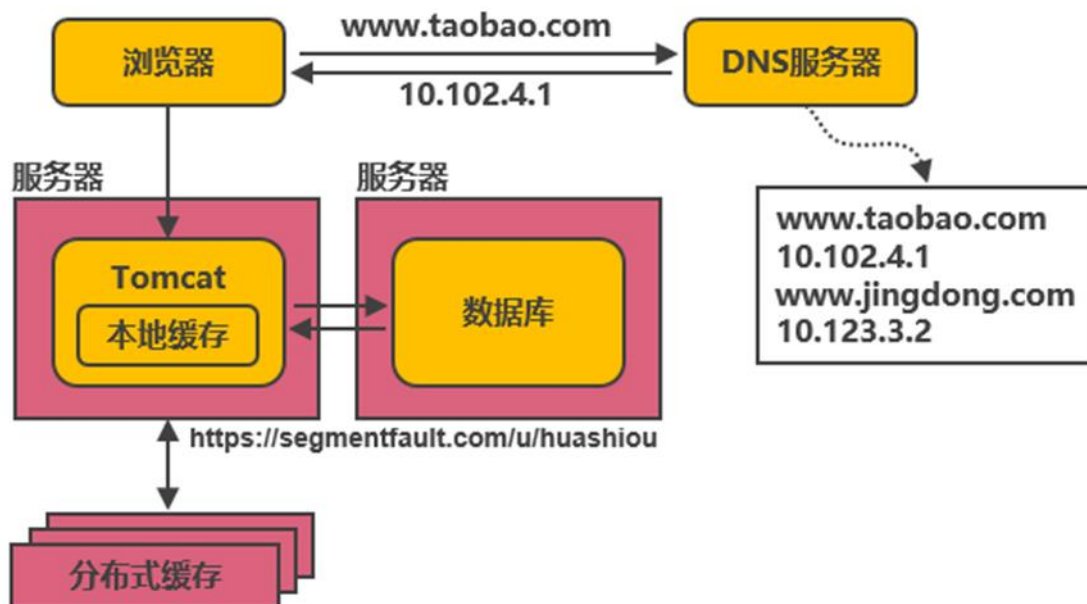
Tomcat 和数据库分别独占服务器资源，显著提高两者各自性能。



随着用户数的增长，并发读写数据库成为瓶颈。

3、第二次演进：引入本地缓存和分布式缓存

在 Tomcat 服务器增加本地缓存，并在外部增加分布式缓存，缓存热门商品信息或热门商品的 HTML 页面等。通过缓存能把绝大多数请求在读写数据库前拦截掉，大大降低数据库压力。

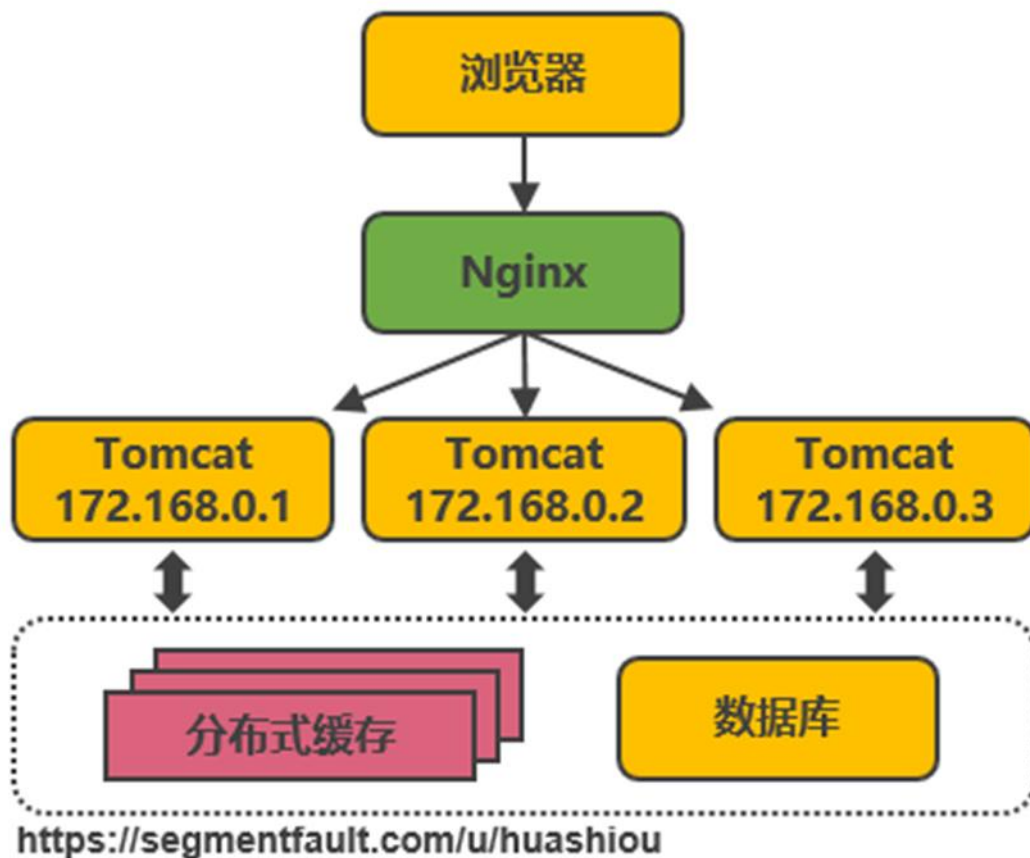


涉及的技术包括：使用 memcached 作为本地缓存，使用 Redis 作为分布式缓存，还会涉及缓存一致性、缓存穿透/击穿、缓存雪崩、热点数据集中失效等问题。

缓存扛住了大部分的访问请求，随着用户数的增长，,并发压力主要落在单机的 Tomcat 上，响应逐渐变慢。

4、第三次演进：引入反向代理实现负载均衡

在多台服务器上分别部署 Tomcat，使用反向代理软件 Nginx 把请求均匀分发到每个 Tomcat 中。假设 Tomcat 最多支持 100 个并发，Nginx 最多支持 50000 个并发，那么理论上 Nginx 把请求分发到 500 个 Tomcat 上，就能抗住 50000 个并发。

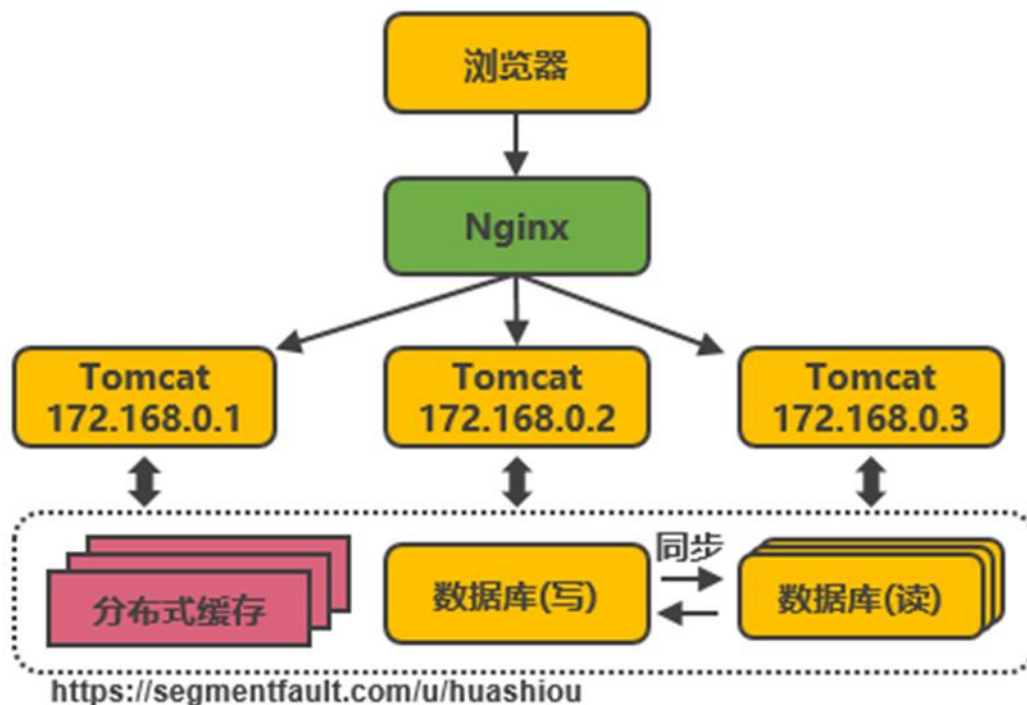


涉及的技术包括：Nginx、HAProxy，两者都是工作在 ISO/OSI 第七层的反向代理软件，主要支持 HTTP 协议,还会涉及 session 共享、文件上传下载的问题。

反向代理使应用服务器可支持的并发量大大增加，但并发量的增长也意味着更多请求穿透到数据库，单机的数据库最终成为瓶颈。

5、第四次演进：数据库读、写分离

把数据库划分为读库和写库，读库可以有多个，通过同步机制把写库的数据同步到读库，对于需要查询最新写入数据场景，可通过在缓存中多写一份，通过缓存获得最新数据。

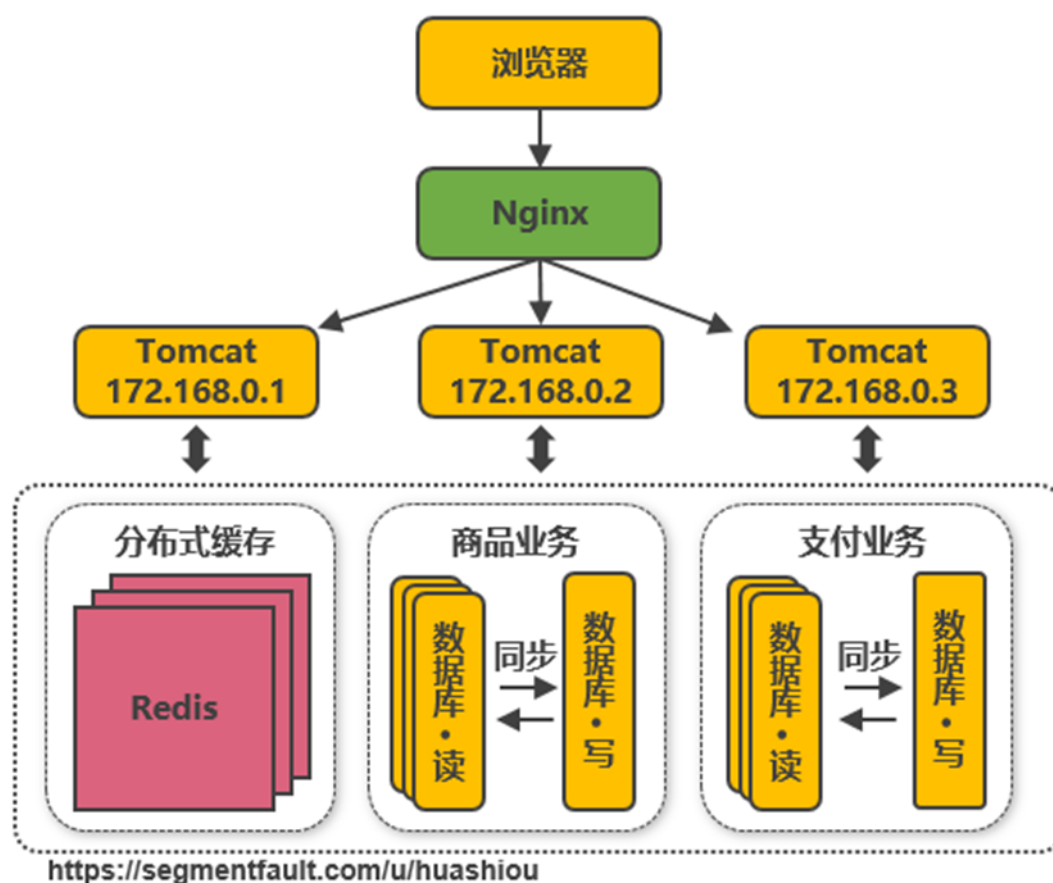


涉及的技术包括：Mycat，它是数据库中间件，可通过它来组织数据库的分离读写和分库分表，客户端通过它来访问下层数据库，还会涉及数据同步，数据一致性的问题。

业务逐渐变多，不同业务之间的访问量差距较大，不同业务直接竞争数据库，相互影响性能。

6、第五次演进：数据库按业务分库

把不同业务的数据保存到不同的数据库中，降低业务之间的资源竞争，对于访问量大的业务，可以部署更多的服务器来支撑。这样同时导致跨业务的表无法直接做关联分析，需要通过其他途径来解决。



随着用户数的增长，单机的写库会逐渐达到性能瓶颈。

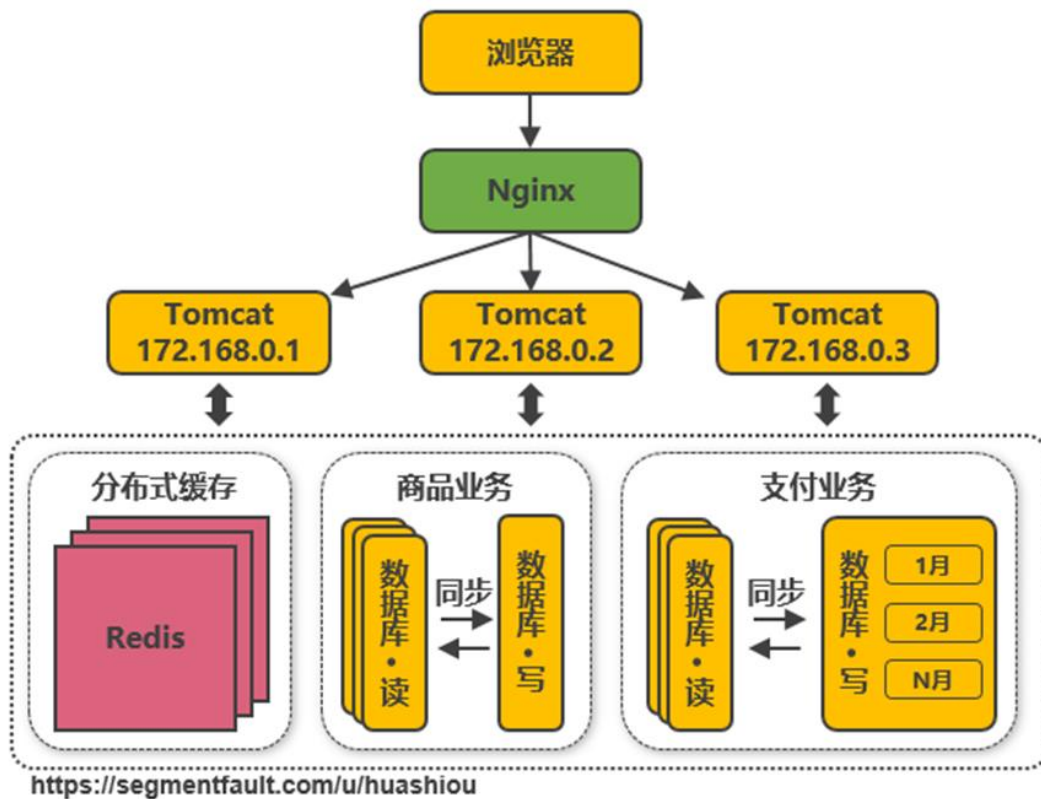
7、第六次演进：把大表拆分为小表

数据库设计采用这种结构时，已经可以称为分布式数据库，但这只是一个逻辑的数据库整体，数据库里不同的组成部分是由不同的组件单独来实现的，如：

- 分库分表的管理和请求分发，由 Mycat 实现（Mycat 支持在大表拆分为小表情况下的访问控制）；

- SQL 的解析由单机的数据库实现；
- 读、写分离可能由网关和消息队列来实现；
- 查询结果的汇总可能由数据库接口层来实现等。

这种架构其实是 MPP（大规模并行处理）架构的一类实现。



- 分布式
- 并行分发

比如针对评论数据，可按照商品 ID 进行 hash，路由到对应的表中存储；针对支付记录，可按照小时创建表，每个小时表继续拆分为小表，使用用户 ID 或记录编号来路由数据。只要实时操作的表数据量足够小，请求能够足够均匀地分发到多台服务器上的小表，那数据库就能通过水平扩展的方式来提高性能。

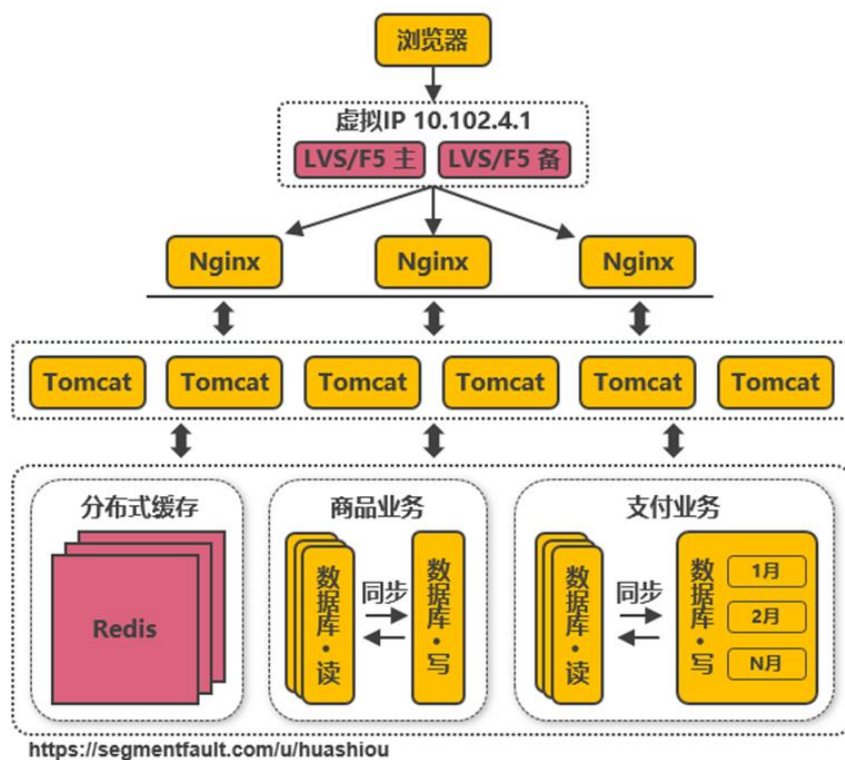
这种做法显著增加了数据库运维的难度，对 DBA 的要求较高!!!

DB 和 Tomcat 都能水平扩展，可支撑的并发大幅提高，随着用户数的增长，最终单机的 Nginx 会成为瓶颈。

8、第七次演进：使用 LVS 或 F5 使多个 Nginx 负载均衡

LVS 是软件，运行在操作系统内核态，可对 TCP 请求或更高层级的网络协议进行转发，因此支持的协议更丰富，并且性能也远高于 Nginx，可假设单机的 LVS 可支持几十万个并发的请求转发；F5 是一种负载均衡硬件，与 LVS 提供的能力类似，性能比 LVS 更高，但价格昂贵。

可使用 keepalived 软件模拟虚拟 IP，然后把虚拟 IP 绑定到多台 LVS 服务器。浏览器访问虚拟 IP 时，会被路由器重定向到真实的 LVS 服务器，当主 LVS 服务器宕机时，keepalived 软件会自动更新路由器中的路由表，把虚拟 IP 重定向到另外一台正常的 LVS 服务器，从而达到 LVS 服务器高可用的效果。



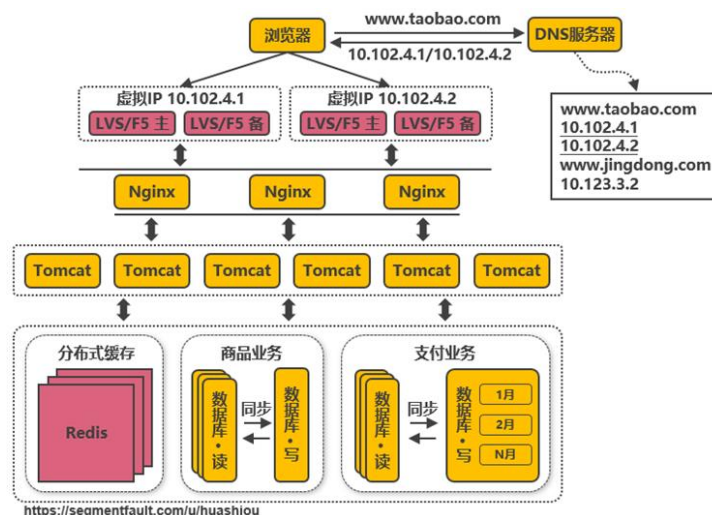
LVS 是单机版的软件，它所在服务器宕机会导致整个后端系统都无法访问，因此需要有备用节点。

此处需要注意的是，从 Nginx 层到 Tomcat 层这样画并不代表全部 Nginx 都转发请求到全部的 Tomcat，在实际使用时，可能会是几个 Nginx 下面接一部分的 Tomcat，这些 Nginx 之间通过 keepalived 实现高可用，其他的 Nginx 接另外的 Tomcat，这样可接入的 Tomcat 数量就能成倍的增加。

由于 LVS 也是单机的，随着并发数增长到几十万时，LVS 服务器最终会达到瓶颈，此时用户数达到千万甚至上亿级别，用户分布在不同的地区，与服务器机房距离不同，导致了访问的延迟会明显不同 Nginx 会成为瓶颈。

9、第八次演进：通过 DNS 轮询实现机房间的负载均衡

在 DNS 服务器中可配置一个域名对应多个 IP 地址，每个 IP 地址对应到不同的机房里的虚拟 IP。当用户访问 `www.taobao.com` 时，DNS 服务器会使用轮询策略或其他策略，来选择某个 IP 供用户访问。

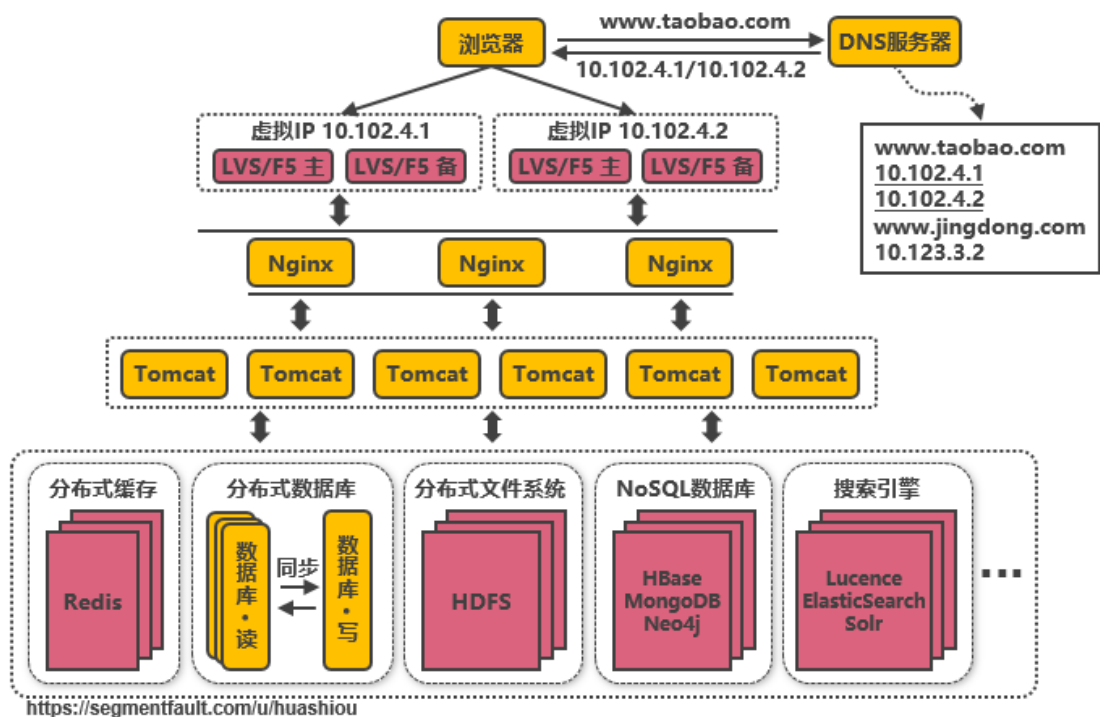


此方式能实现机房间的负载均衡。至此，系统可做到机房级别的水平扩展，千万级到亿级的并发量都可通过增加机房来解决，系统入口处的请求并发量不再是问题。

当数据多到一定规模时，数据库就不适用于复杂的查询了，往往只能满足普通查询的场景。对于统计报表场景，在数据量大时不一定能跑出结果，而且在跑复杂查询时会导致其他查询变慢，对于全文检索、可变数据结构等场景，数据库天生不适用。

10、第九次演进：引入 NoSQL 数据库和搜索引擎等技术

针对特定的场景，引入合适的解决方案。如对于海量文件存储，可通过分布式文件系统 HDFS 解决，对于 key/value 类型的数据，可通过 HBase 和 Redis 等方案解决，对于全文检索场景，可通过搜索引擎如 ElasticSearch 解决，对于多维分析场景，可通过 Kylin 或 Druid 等方案解决。

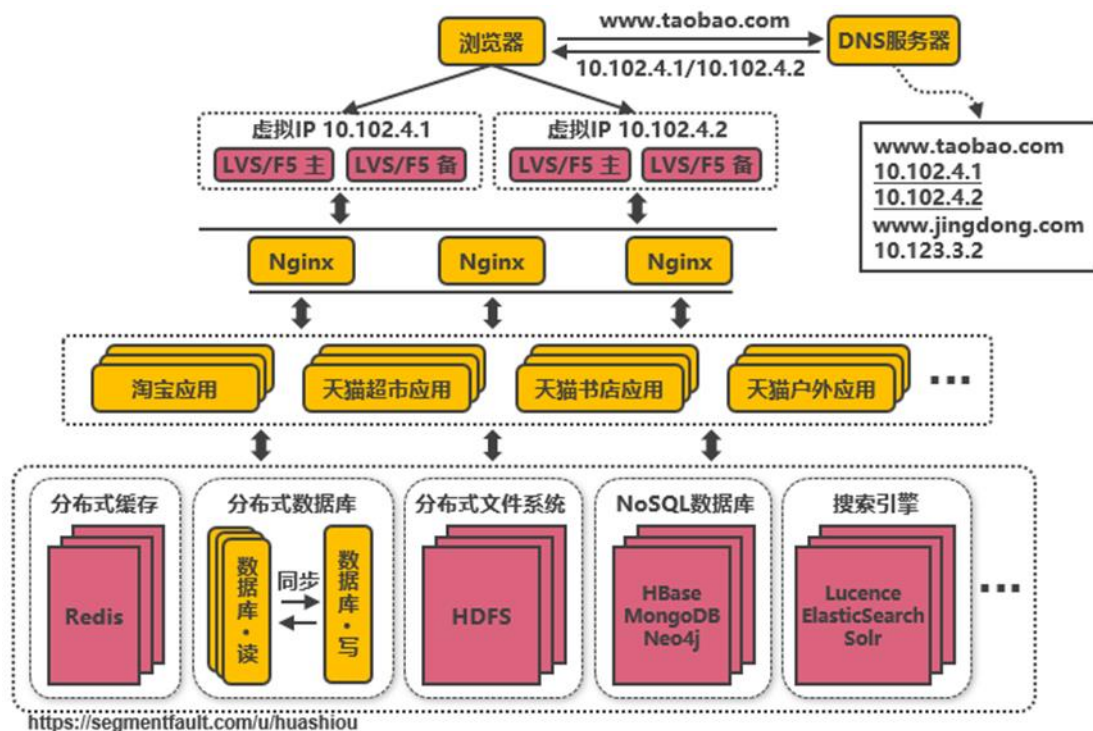


当然，引入更多组件同时会提高系统的复杂度，不同的组件保存的数据需要同步，需要考虑一致性的问题，需要有更多的运维手段来管理这些组件等。

引入更多组件解决了丰富的需求，业务维度能够极大扩充，随之而来的是一个应用中包含了太多的业务代码，业务的升级迭代变得困难。

11、第十次演进：大应用拆分为小应用

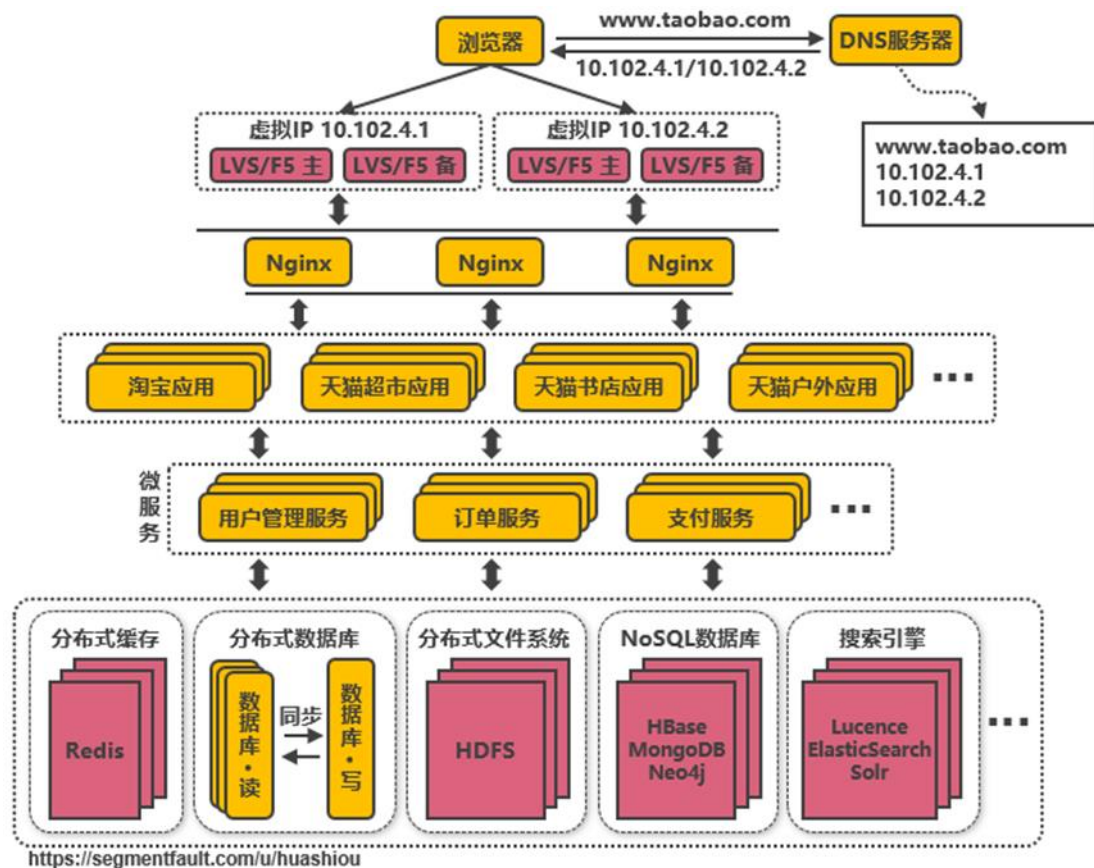
按照业务板块来划分应用代码，使单个应用的职责更清晰，相互之间可以做到独立升级迭代。这时候应用之间可能会涉及到一些公共配置，可以通过分布式配置中心 Zookeeper 来解决。



不同应用之间存在共用的模块，由应用单独管理会导致相同代码存在多份，导致公共功能升级时全部应用代码都要跟着升级。

12、第十一次演进：复用的功能抽离成微服务

如用户管理、订单、支付、鉴权等功能在多个应用中都存在，那么可以把这些功能的代码单独抽取出来形成一个单独的服务来管理，这样的服务就是所谓的微服务，应用和服务之间通过 HTTP、TCP 或 RPC 请求等多种方式来访问公共服务，每个单独的服务都可以由单独的团队来管理。

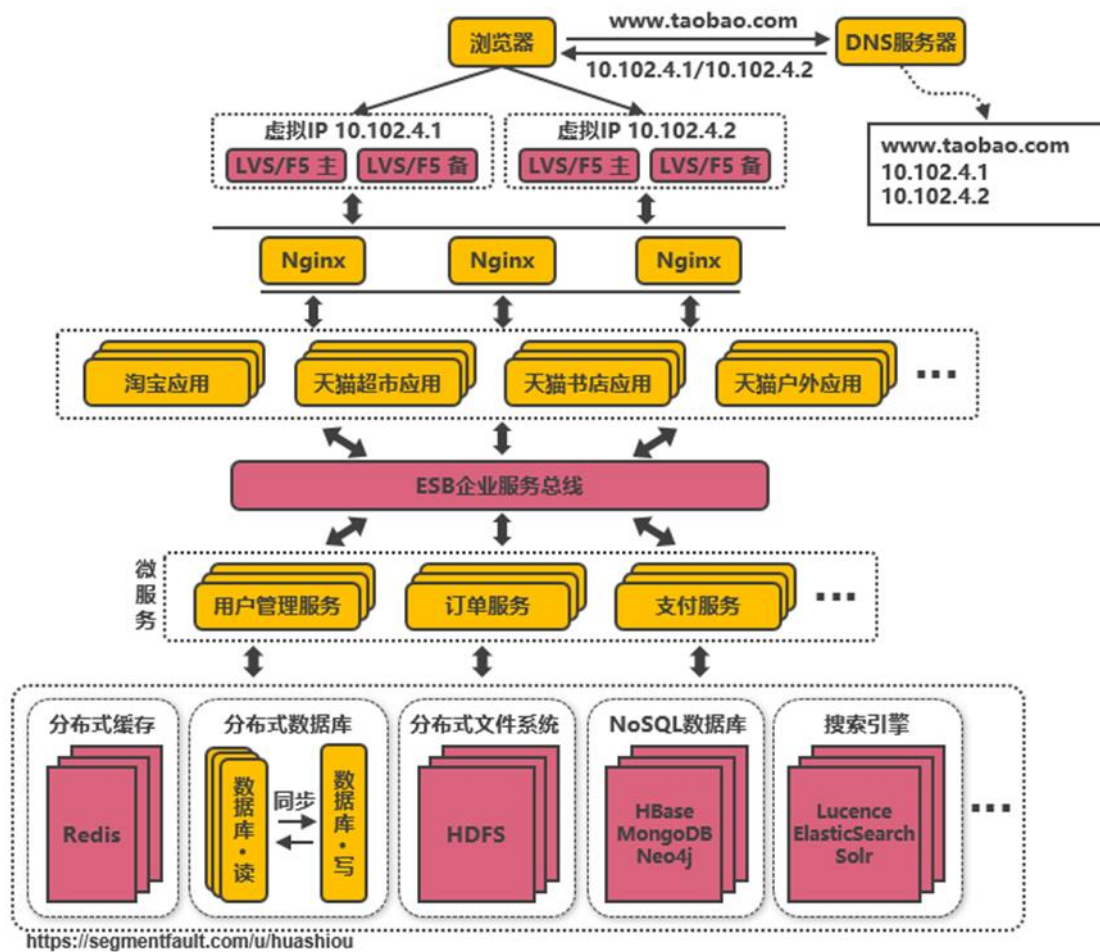


此外，可以通过 Dubbo、SpringCloud 等框架实现服务治理、限流、熔断、降级等功能，提高服务的稳定性和可用性。

不同服务的接口访问方式不同，应用代码需要适配多种访问方式才能使用服务。此外，应用访问服务，服务之间也可能相互访问，调用链将会变得非常复杂，逻辑变得混乱。

13、第十二次演进：引入企业服务总线 ESB 屏蔽服务接口的访问差异

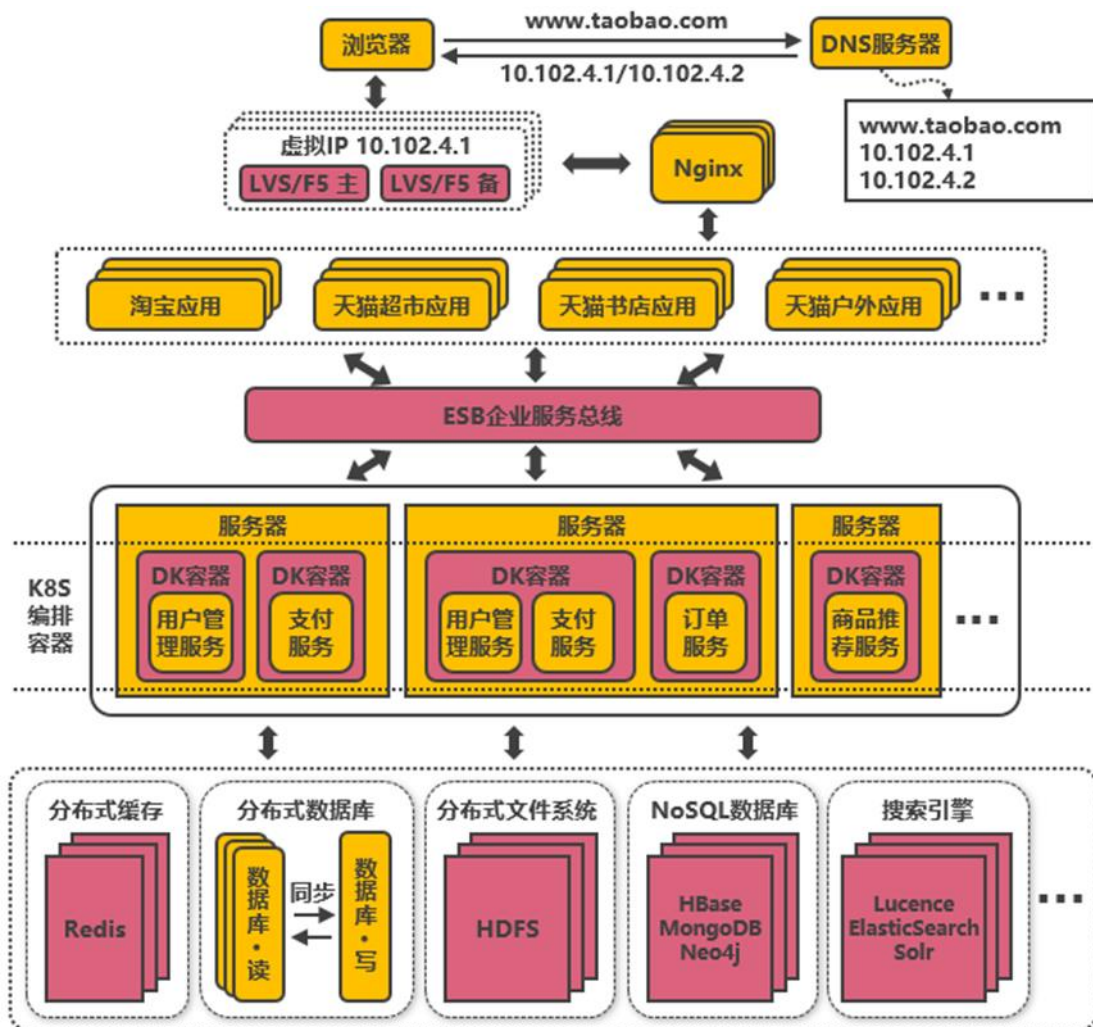
通过 ESB 统一进行访问协议转换，应用统一通过 ESB 来访问后端服务，服务与服务之间也通过 ESB 来相互调用，以此降低系统的耦合程度。这种单个应用拆分为多个应用，公共服务单独抽取出来来管理，并使用企业消息总线来解除服务之间耦合问题的架构，就是所谓的 SOA（面向服务）架构。



业务不断发展，应用和服务都会不断变多，应用和服务的部署变得复杂，同一台服务器上部署多个服务还要解决运行环境冲突的问题。此外，对于如大促这类需要动态扩缩容的场景，需要水平扩展服务的性能，就需要在新增的服务上准备运行环境，部署服务等，运维将变得十分困难。

14、第十三次演进：引入容器化技术实现运行环境隔离与动态服务管理

目前最流行的容器化技术是 Docker，最流行的容器管理服务是 Kubernetes (K8S)，应用/服务可以打包为 Docker 镜像，通过 K8S 来动态分发和部署镜像。Docker 镜像可理解为一个能运行应用/服务的最小的操作系统，里面放着应用/服务的运行代码，运行环境根据实际的需要设置好。把整个“操作系统”打包为一个镜像后，就可以分发到需要部署相关服务的机器上，直接启动 Docker 镜像就可以把服务起起来，使服务的部署和运维变得简单。



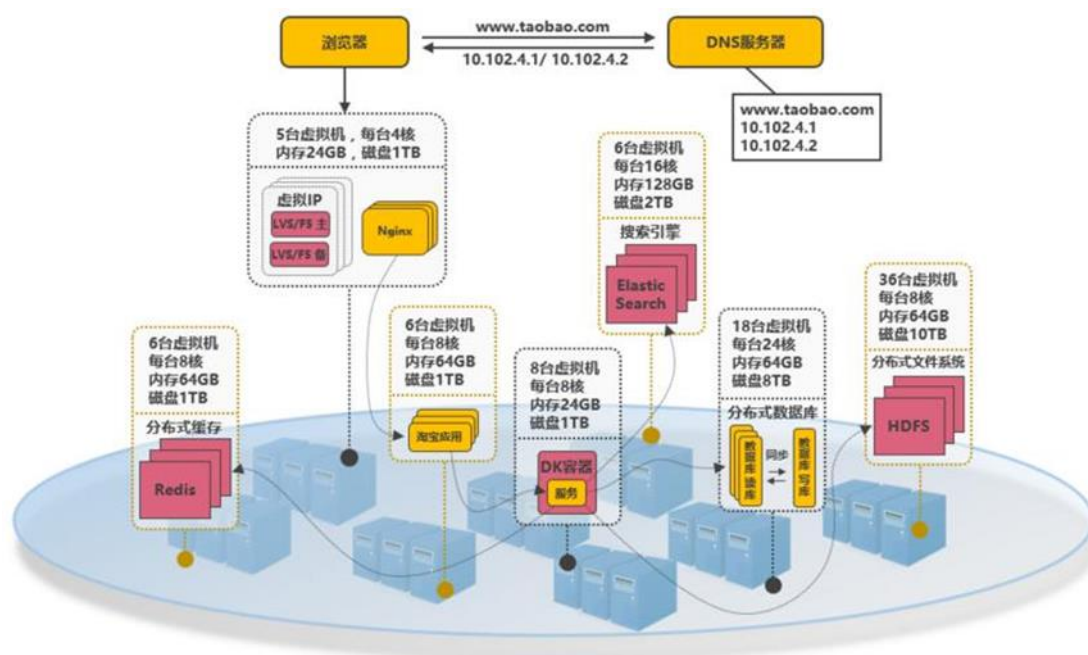
<https://segmentfault.com/u/huashiou>

大促之前，可以在现有的机器集群上划分出服务器来启动 Docker 镜像，增强服务的性能，大促过后就可以关闭镜像，对机器上的其他服务不造成影响。

使用容器化技术后服务动态扩缩容问题得以解决，但是机器还是需要公司自身来管理，在非大促的时候，还是需要闲置着大量的机器资源来应对大促，机器自身成本和运维成本都极高，资源利用率低。

15、第十四次演进：以云平台承载系统

系统可部署到公有云上，利用公有云的海量机器资源，解决动态硬件资源的问题，在大促的时间段里，在云平台中临时申请更多的资源，结合 Docker 和 K8S 来快速部署服务，在大促结束后释放资源，真正做到按需付费，资源利用率大大提高，同时大大降低了运维成本。



- 以上所提到的从高并发访问问题，到服务的架构和系统实施的层面都有了各自的解决方案。

- 针对更复杂的问题，还需考虑诸如跨机房数据同步、分布式事务实现等实际问题。

16、总结

架构的演进总结：从一百个并发到千万级并发

每个演进阶段会遇到的相关技术

分布式	系统中的多个模块在不同服务器上部署，即可称为分布式系统，如Tomcat和数据库分别部署在不同的服务器上，或两个相同功能的Tomcat分别部署在不同服务器上
高可用	系统中部分节点失效时，其他节点能够接替它继续提供服务，则可认为系统具有高可用性
集群	一个特定领域的软件部署在多台服务器上并作为一个整体提供一类服务，这个整体称为集群。如Zookeeper中的Master和Slave分别部署在多台服务器上，共同组成一个整体提供集中配置服务。在常见的集群中，客户端往往能够连接任意一个节点获得服务，并且当集群中一个节点掉线时，其他节点往往能够自动的接替它继续提供服务，这时候说明集群具有高可用性
负载均衡	请求发送到系统时，通过某些方式把请求均匀分发到多个节点上，使系统中每个节点能够均匀的处理请求负载，则可认为系统是负载均衡的
正向代理和反向代理	系统内部要访问外部网络时，统一通过一个代理服务器把请求转发出去，在外部网络看来就是代理服务器发起的访问，此时代理服务器实现的是正向代理；当外部请求进入系统时，代理服务器把该请求转发到系统中的某台服务器上，对外部请求来说，与之交互的只有代理服务器，此时代理服务器实现的是反向代理。简单来说，正向代理是代理服务器代替系统内部来访问外部网络的过程，反向代理是外部请求访问系统时通过代理服务器转发到内部服务器的过程。

17、秒杀架构设计思路

将请求拦截在系统上游，降低下游压力：秒杀系统特点是并发量极大，但实际秒杀成功的请求数量却很少，所以如果不在前端拦截很可能造成数据库读写锁冲突，甚至导致死锁，最终请求超时。

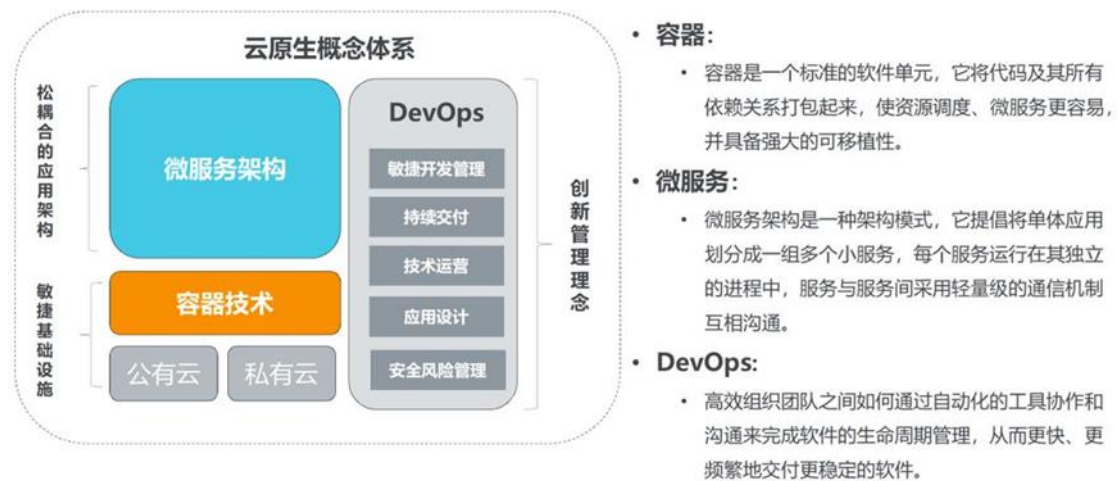
充分利用缓存（Redi）：利用缓存可极大提高系统读写速度。

消息中间件（ActiveMQ、Kafka 等）：消息队列可以削峰，将拦截大量并发请求，这是一个异步处理过程，后台业务根据自己的处理能力，从消息队列中主动的拉取请求消息进行业务处理。

限流，削峰，异步处理，内存缓存，可拓展

18、第十五次演进：…

云原生是基于分布部署和统一运管的分布式云，以容器、微服务、DevOps 等技术为基础建立的一套云技术产品体系。



充分利用和发挥云平台的弹性+分布式优势，提升云上资源利用率、缩短开发周期。

四、信息系统架构复杂性度量

1、复杂性度量问题的提出

体系结构技术是进行信息系统设计的重要技术,其复杂性直接决定着建设信息系统的复杂性。

2、信息系统体系架构复杂性度量

体系结构是信息系统的基本结构，它的复杂性直接影响着系统的复杂性。

在体系结构设计阶段有效控制复杂性，仅可以减小系统实现的难度、降低成本，而且对提高系统可靠性等也有一定的帮助。因此，对体系结构的复杂性进行度量和评价，对于系统的设计和决策非常重要。通过对体系结构复杂性的研究，对复杂性给出合适的定义和度量方法，就可以采取有效的措施来降低系统复杂性，减小复杂性所带来的不利影响。

3、评估信息系统体系架构复杂性的必要性（以软件工程为例）

McCabe 建议对复杂度超过 10 的程序分成几个子程序以减少程序错误。

4、信息系统体系架构复杂性

体系结构的复杂性包括结构复杂性和动态复杂性。

（1）结构复杂性

系统中的层次结构给系统建设带来的难度。

（2）动态复杂性

由于不同节点之间相互关系的多样性和差异性以及动态运行过程中的不确定性所带来的建设和维护的难度。

它们从不同的层次反映了体系结构的复杂性，并且都决定着系统复杂性。

等级层次结构是复杂性的主要根源之一，复杂性与系统的等级层次结构有直接关系，层次越多，越容易产生复杂性。对于系统结构复杂性的度量就可以转换为对体系结构层次结构复杂性的度量。

5、经典的结构化程序复杂性度量方法

(1) Halstead 复杂性度量

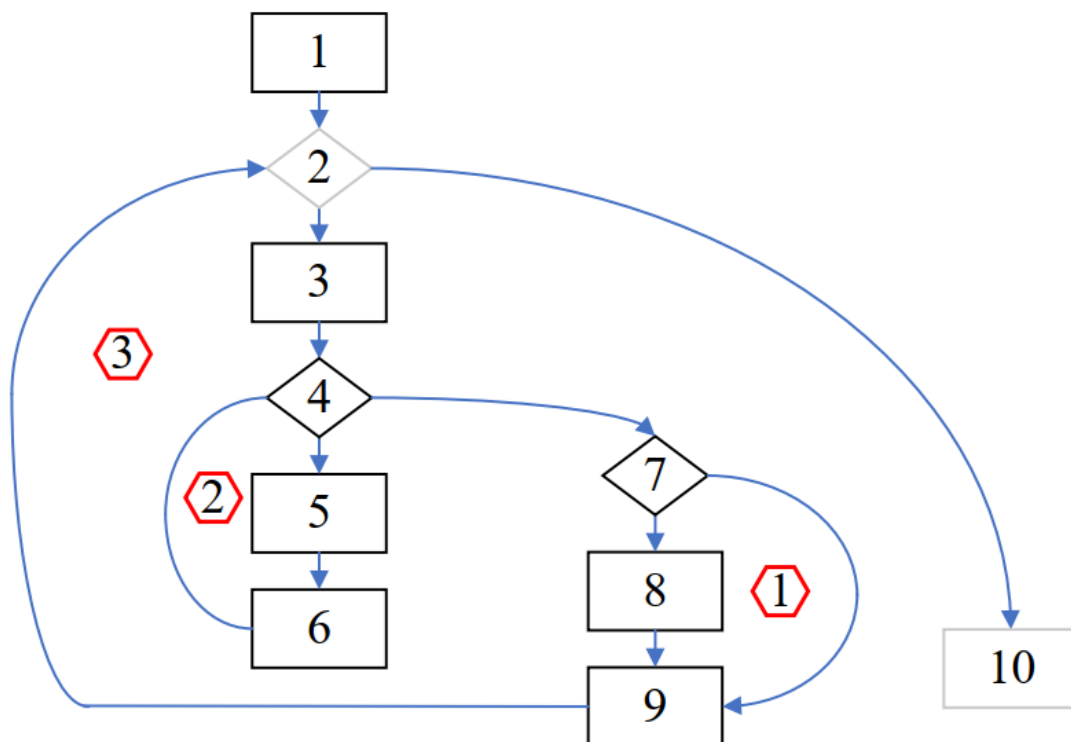
Halstead 把程序看成由可执行的代码行的词汇（操作符和操作数）组成的符号序列。设 n_1 表示程序中不同运算符的个数， n_2 表示程序中不同操作数的个数，令 H 表示程序的预测长度，Halstead 给出 H 的计算公式为：

$$H = n_1 \log_2 n_1 + n_2 \log_2 n_2$$

Halstead 的重要结论之一是：程序的实际长度 N 与预测长度非常接近。

(2) McCabe 度量法

M McCabe 度量用程序流图的圈数（cyclomatic number）测量程序的复杂性，并基于程序控制论和图论提出了经典的 McCabe 圈复杂性度量理论。



复杂度：

流图中的区域数：4（上图中用方框标注的）

流图 G 的**环形复杂度** $V(G) = E - N + 2$ ，其中 E 是流图中边的条数，N 是结点数。本例中：12-10+2=4

流图 G 的**环形复杂度** $V(G) = P + 1$ ，其中 P 是流图中判定结点的数目。本例中 3+1=4（判定节点是 2， 4， 7）